Measurement Error Models

General Principles

Measurement error refers to the variability in the measurement of a variable, and measurement error can be generated by several factors, such as sampling bias, censoring bias, and group size heterogeneity. It is an important consideration in many fields, including statistics, economics, and engineering, where accurate measurements are crucial for making informed decisions. To account for measurement error, we can use a *measurement error model*. This model assumes that the measurement of a variable is subject to an error, which can be modeled using a probability distribution. The model can be used to estimate the parameters of the measurement error distribution, such as the mean and variance, and to make predictions about the measurements based on the estimated parameters. Measurement error models are *composed models* (i.e., models with sub-models) that evaluate different generative processes, starting with the measurement error process, which is then used to generate the observed data.

Example

Below is an example code snippet demonstrating a Bayesian measurement error model using the Bayesian Inference (BI) package. The data consist of three continuous variables (marriage rate, divorce rate, age), and the goal is to estimate the effect of age and marriage rate on the divorce rate while considering that the divorce rate has a measurement error. This example is based on McElreath (2018).

Python

```
from BI import bi, jnp

# Setup device--------------------------------------------------
m = bi(platform='cpu')

# Import Data & Data Manipulation ------------------------------------------------
```

```python
# Import
data_path = m.load.WaffleDivorce(only_path=True)
m.data(data_path, sep=';')
m.scale(['MedianAgeMarriage', 'Marriage']) # Scale
dat = dict(
    D_obs = m.z_score(m.df['Divorce'].values),
    D_sd = jnp.array(m.df['Divorce SE'].values / m.df['Divorce'].std()),
    A = jnp.array(m.df['MedianAgeMarriage'].values),
    M = jnp.array(m.df['Marriage'].values),
    N = m.df.shape[0]
)
m.data_on_model = dat # Send to model (convert to jax array)

# Define model ------------------------------------------------
def model(D_obs, D_sd, A, N, M):
    a = m.dist.normal(0, 0.2, name = 'a')
    beta = m.dist.normal(0, 0.5, name = 'beta')
    eta = m.dist.normal(0, 0.5, name = 'eta')
    s = m.dist.exponential(1, name = 's')
    mu = a + beta * A + eta * M
    D_true = m.dist.normal(mu, s, name = 'D_true')
    m.dist.normal(D_true , D_sd, obs = D_obs)

# Run MCMC ----------------------------------------------------
m.fit(model, progress_bar=False)  # Optimize model parameters through MCMC sampling

# Summary -----------------------------------------------------
m.summary() # Get posterior distributions
```

```
jax.local_device_count 16
```

```
arviz - WARNING - Shape validation failed: input_shape: (1, 500), minimum_shape: (chains=2, d
```

	mean	sd	hdi_5.5%	hdi_94.5%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
D_true[0]	1.21	0.36	0.68	1.84	0.01	0.01	729.75	368.04	NaN
D_true[1]	0.69	0.55	-0.10	1.52	0.02	0.03	725.94	272.71	NaN
D_true[2]	0.46	0.33	-0.04	1.04	0.01	0.01	719.10	358.85	NaN
D_true[3]	1.43	0.48	0.56	2.06	0.02	0.02	580.25	351.02	NaN
D_true[4]	-0.91	0.14	-1.13	-0.68	0.00	0.01	891.55	302.34	NaN
D_true[5]	0.66	0.39	-0.02	1.26	0.01	0.01	725.09	500.52	NaN

	mean	sd	hdi_5.5%	hdi_94.5%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
D_true[6]	-1.39	0.38	-1.93	-0.76	0.01	0.02	687.87	332.33	NaN
D_true[7]	-0.36	0.50	-1.14	0.38	0.02	0.03	717.11	458.50	NaN
D_true[8]	-1.89	0.62	-2.96	-1.03	0.03	0.03	378.27	367.44	NaN
D_true[9]	-0.63	0.16	-0.89	-0.38	0.01	0.01	814.70	310.37	NaN
D_true[10]	0.77	0.29	0.28	1.19	0.01	0.01	549.81	397.25	NaN
D_true[11]	-0.56	0.50	-1.29	0.28	0.02	0.02	708.73	321.38	NaN
D_true[12]	0.15	0.57	-0.64	1.13	0.02	0.03	537.17	294.58	NaN
D_true[13]	-0.87	0.25	-1.27	-0.47	0.01	0.01	420.65	230.93	NaN
D_true[14]	0.56	0.28	0.15	1.00	0.01	0.01	673.63	399.55	NaN
D_true[15]	0.29	0.38	-0.25	1.02	0.01	0.02	830.56	346.12	NaN
D_true[16]	0.50	0.43	-0.24	1.10	0.02	0.02	635.29	288.01	NaN
D_true[17]	1.29	0.33	0.80	1.87	0.01	0.01	862.91	348.34	NaN
D_true[18]	0.45	0.38	-0.21	0.97	0.01	0.02	817.61	347.16	NaN
D_true[19]	0.44	0.56	-0.34	1.39	0.03	0.02	354.76	288.24	NaN
D_true[20]	-0.57	0.31	-1.04	-0.08	0.01	0.02	545.94	303.79	NaN
D_true[21]	-1.11	0.25	-1.51	-0.73	0.01	0.01	1304.68	352.86	NaN
D_true[22]	-0.27	0.25	-0.69	0.12	0.01	0.01	942.98	244.36	NaN
D_true[23]	-1.01	0.27	-1.40	-0.55	0.01	0.01	434.31	365.09	NaN
D_true[24]	0.43	0.40	-0.19	1.06	0.02	0.02	651.82	305.99	NaN
D_true[25]	-0.03	0.29	-0.53	0.40	0.01	0.01	717.47	360.28	NaN
D_true[26]	-0.07	0.53	-0.91	0.80	0.03	0.02	446.11	421.86	NaN
D_true[27]	-0.15	0.42	-0.72	0.57	0.02	0.02	851.71	374.38	NaN
D_true[28]	-0.27	0.51	-1.05	0.50	0.02	0.02	505.27	353.08	NaN
D_true[29]	-1.82	0.25	-2.16	-1.37	0.01	0.01	629.54	312.60	NaN
D_true[30]	0.18	0.46	-0.63	0.85	0.01	0.03	1213.78	271.91	NaN
D_true[31]	-1.67	0.17	-1.92	-1.40	0.01	0.01	557.46	408.74	NaN
D_true[32]	0.12	0.24	-0.22	0.54	0.01	0.01	565.48	407.54	NaN
D_true[33]	-0.09	0.49	-0.89	0.64	0.02	0.03	549.62	342.20	NaN
D_true[34]	-0.12	0.23	-0.44	0.28	0.01	0.01	908.80	344.08	NaN
D_true[35]	1.28	0.43	0.67	2.05	0.02	0.02	595.77	424.30	NaN
D_true[36]	0.25	0.32	-0.24	0.74	0.01	0.02	859.97	332.33	NaN
D_true[37]	-1.04	0.21	-1.38	-0.74	0.01	0.01	701.84	473.99	NaN
D_true[38]	-0.92	0.56	-1.87	-0.12	0.02	0.03	562.98	385.44	NaN
D_true[39]	-0.71	0.30	-1.25	-0.29	0.01	0.01	649.48	419.43	NaN
D_true[40]	0.24	0.53	-0.71	1.03	0.02	0.03	753.76	321.53	NaN
D_true[41]	0.76	0.33	0.26	1.25	0.02	0.01	348.37	332.48	NaN
D_true[42]	0.20	0.17	-0.02	0.54	0.01	0.01	699.36	330.63	NaN
D_true[43]	0.81	0.42	0.14	1.49	0.02	0.01	570.84	414.15	NaN
D_true[44]	-0.39	0.54	-1.29	0.47	0.02	0.03	905.00	309.26	NaN
D_true[45]	-0.40	0.23	-0.77	-0.04	0.01	0.01	642.94	322.24	NaN
D_true[46]	0.12	0.30	-0.35	0.59	0.01	0.01	944.14	372.06	NaN

	mean	sd	hdi_5.5%	hdi_94.5%	mcse_mean	mcse_sd	ess_bulk	ess_tail	r_hat
D_true[47]	0.59	0.49	-0.21	1.30	0.02	0.03	1078.81	281.64	NaN
D_true[48]	-0.64	0.29	-1.15	-0.23	0.01	0.02	598.08	368.73	NaN
D_true[49]	0.86	0.60	-0.04	1.83	0.03	0.02	399.66	341.66	NaN
a	-0.05	0.10	-0.22	0.10	0.00	0.00	430.67	295.70	NaN
beta	-0.63	0.17	-0.87	-0.37	0.01	0.01	278.18	231.62	NaN
eta	0.05	0.17	-0.22	0.29	0.01	0.01	266.00	382.42	NaN
s	0.60	0.11	0.42	0.74	0.01	0.01	102.30	333.30	NaN

R

```
library(BayesianInference)
jnp = reticulate::import('jax.numpy')

# Setup platform-----------------------------------------------
m=importBI(platform='cpu')

# Import data -------------------------------------------------
m$data(paste(system.file(package = "BayesianInference"),"/data/WaffleDivorce.csv", sep = ''))

m$scale(list('MedianAgeMarriage', 'Marriage'))

m$data_on_model$D_obs = m$z_score(jnp$array(m$df['Divorce']))
m$data_on_model$D_sd = jnp$array(m$df['Divorce SE']) / sd(unlist(m$df['Divorce']))
m$data_on_model$A = jnp$array(m$df['MedianAgeMarriage'])
m$data_on_model$M = jnp$array(m$df['Marriage'])
m$data_on_model$N = as.integer(nrow(m$df))

# Define model ------------------------------------------------
model <- function(D_obs, D_sd, A, N, M){
  a = bi.dist.normal(0, 0.2, name = 'a')
  beta = bi.dist.normal(0, 0.5, name = 'beta')
  eta = bi.dist.normal(0, 0.5, name = 'eta')
  s = bi.dist.exponential(1, name = 's')
  mu = a + beta * A + eta * M
  D_true = bi.dist.normal(mu, s, name = 'D_true')
  bi.dist.normal(D_true , D_sd, obs = D_obs)
}

# Run MCMC ----------------------------------------------------
```

```
m$fit(model) # Optimize model parameters through MCMC sampling

# Summary ----------------------------------------------
m$summary() # Get posterior distribution
```

Julia

```
using BayesianInference

# Setup device-----------------------------------------------
m = importBI(platform="cpu")

# Import Data & Data Manipulation ----------------------------------------------
# Import
data_path = m.load.WaffleDivorce(only_path=true)
m.data(data_path, sep=";")
m.scale(["MedianAgeMarriage", "Marriage"]) # Scale
dat = pydict(
    D_obs = m.z_score(m.df["Divorce"].values),
    D_sd = jnp.array(m.df["Divorce SE"].values / m.df["Divorce"].std()),
    A = jnp.array(m.df["MedianAgeMarriage"].values),
    M = jnp.array(m.df["Marriage"].values),
    N = m.df.shape[0]
)
m.data_on_model = dat # Send to model (convert to jax array)

# Define model ----------------------------------------------
@BI function model(D_obs, D_sd, A, N, M)
    a = m.dist.normal(0, 0.2, name = "a")
    beta = m.dist.normal(0, 0.5, name = "beta")
    eta = m.dist.normal(0, 0.5, name = "eta")
    s = m.dist.exponential(1, name = "s")
    mu = a + beta * A + eta * M
    D_true = m.dist.normal(mu, s, name = "D_true")
    m.dist.normal(D_true , D_sd, obs = D_obs)


end

# Run mcmc ----------------------------------------------
m.fit(model)  # Optimize model parameters through MCMC sampling
```

```
# Summary ----------------------------------------------
m.summary() # Get posterior distributions
```

Mathematical Details

Bayesian formulation

$$D_i^* \sim \text{Normal}(D_i, \varsigma_i)$$

$$D_i \sim \text{Normal}(\mu_i, \sigma)$$

$$\mu_i = \alpha + \beta A_i + \eta M_i$$

$$\sigma \sim \text{Normal}(1)$$

where:

- D_i^* is the observed divorce rate.
- D_i is the true divorce rate.
- μ_i is the mean of the true divorce rate.
- σ is the standard deviation of the true divorce rate.
- α is the intercept term.
- β is the regression coefficient for age.
- η is the regression coefficient for marriage rate.

Notes

> **Note**
>
> This is an approach that can be extended to any kind of model previously described. For example, one could generate a Bernoulli measurement error model by generating a process for the probabilities of success and failure. We can even go further by potentially having an error rate that is present only in one of the two outcomes.

Reference(s)

McElreath, Richard. 2018. *Statistical Rethinking: A Bayesian course with examples in R and Stan*. Chapman; Hall/CRC.